

1. Which list implementation did you use as the snake's body?
 - I choose the LinkedList implementation to use as the snake body.
2. Why did you select that implementation?
 - Because the snake always add new head at index 0 every turn. Linked list can do this very fast in $O(1)$ time, but array need to shift every elements which take $O(N)$ time. Also, for the array, we have to resize the array when it reaches to the max capacity, which make the run time more complex.
3. Which list operations occur during a normal snake movement?
 - During normal move, we call insert at index 0 to add the new head, and call remove at the last index to delete the tail.
4. Which operations occur when the snake eats food?
 - When snake eat the food, we only call insert at index 0 for new head. We do not call remove for the tail, so the snake grow by one segment.
5. What is the worst-case runtime for one movement step?
 - The worst-case runtime for one move step is $O(N)$. Even if insert at the front is fast, we still have to check body collision which take $O(N)$ time.
6. How does collision detection affect the runtime of a game step?
 - Collision detection make the game step slower. Because the game must use a loop to inspect the list for all occupied positions, it add $O(N)$ work to every turn to check if head hit the body.
7. What advantages does the array-based implementation provide?
 - The array-based give very fast access to index. Also it does not need to create and delete new node on the heap every single move, so it save some memory overhead.
8. What advantages does the linked implementation provide?
 - The linked list advantage is very fast insert and remove at the front. It only do pointer hookup and not need to shift all the elements or resize the array when capacity is full.
9. Which implementation would you choose if indexed access were the most common operation?
 - I will choose ArrayList. Because array provide the fastest time to access any index directly.
10. Which implementation would you choose if insertion and removal at the front were the most common operations?
 - I will choose LinkedList. Because insert and remove at the front is $O(1)$ time, and it do not need to shift any other data backward or forward

